

Practical Week-6

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
priyanka@DESKTOP-HOISO7: x + v
GNU nano 8.7.1 src/fifo_server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "/tmp/client_fifo"
#define FIFO2 "/tmp/server_fifo"

int main() {
    char message[100];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server waiting for client...\n");

    int read_fd = open(FIFO1, O_RDONLY);
    read(read_fd, message, sizeof(message));

    printf("Server received: %s\n", message);

    close(read_fd);

    int write_fd = open(FIFO2, O_WRONLY);

    char response[] = "Message processed by server";
    write(write_fd, response, strlen(response) + 1);

    close(write_fd);

    unlink(FIFO1);
    unlink(FIFO2);

    printf("Server response sent.\n");

    return 0;
}
```

Client Creation:

```
priyanka@DESKTOP-HOISO7: x + v
GNU nano 8.7.1 src/fifo_client.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "/tmp/client_fifo"
#define FIFO2 "/tmp/server_fifo"

int main() {
    char message[] = "Hello Server";
    char response[100];

    int write_fd = open(FIFO1, O_WRONLY);

    write(write_fd, message, sizeof(message));

    close(write_fd);

    int read_fd = open(FIFO2, O_RDONLY);

    read(read_fd, response, sizeof(response));

    printf("Server response: %s\n", response);

    close(read_fd);

    return 0;
}
```

Output:

```
priyanka@DESKTOP-HOISO7: x + v - □ x
priyanka@DESKTOP-HOISO7:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$ nano src/fifo_server.c
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$ nano src/fifo_client.c
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$ gcc src/fifo_server.c -o bin/fifo_serv
er
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$ gcc src/fifo_client.c -o bin/fifo_clie
nt
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$ ./bin/fifo_server
Server waiting for client...
Server received: Hello Server
Server response sent.
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$ |

priyanka@DESKTOP-HOISO7:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$ ./bin/fifo_client
Server response: Message processed by server
priyanka@DESKTOP-HOISO7:~/OS_Skill/Week-4$
```

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
GNU nano 8.7.1 src/signal_handler.c *
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");

    else if (sig == SIGTERM)
        printf("SIGTERM received!\n");

    else if (sig == SIGUSR1)
        printf("SIGUSR1 received!\n");
}

int main() {
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Program running...\n");
    printf("PID: %d\n", getpid());
    printf("Send signals using kill command.\n");

    while (1) {
        sleep(1);
    }

    return 0;
}
```

Output:

```
priyanka@DESKTOP-HOISO7A:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ gcc src/signal_handler.c -o bin/signal_handler
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ./bin/signal_handler
Program running...
PID: 890
Send signals using kill command.
^C
SIGINT received!
SIGUSR1 received!
^Z
[1]+  Stopped                  ./bin/signal_handler
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$

priyanka@DESKTOP-HOISO7A:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ kill -SIGUSR1 890
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$
```